Measurement-Indexed Counting:

An Executable Study of OpenAI's #3-SAT Algorithm, Semiring Component Caches, and the Limits of Oblivious Sampling

Chelsea A. McElveen
LogNovations Holdings LLC, San Francisco, CA
chelsea.anne.mcelveen@gmail.com

28 October 2026

Abstract

We study exact model counting through one device: indexing the assignments of a formula by a linear measurement of the assignment. Three regimes span the range from useless to practical. (i) With no measurement, an oblivious sampler that evaluates a formula on a set of assignments fixed before the clauses are read cannot decide satisfiability; we state the elementary lower bounds and defer the full treatment, which corrects the author's own earlier claims, to a companion note. (ii) The algorithm of OpenAI (October 2026) that refutes the Exponential Time Hypothesis counts the models of a bounded-occurrence 3-CNF by deforming the formula algebraically and indexing each group of variables by a few bounded integer measurements. We give an exact, executable model of that pipeline—clause polynomials, replacement forms, measurement designs, the measurement-tuple identity, the Taylor and inclusion–exclusion identity, Hermite interpolation, and the paper's own construction of designs at small parameters—verified against enumeration and against an independent symbolic expansion at $n \leq 14$. Two structural facts follow. Under the paper's random schedules the within-group clause hypergraph is critical at $r = 2$ (largest component $\Theta(n^{2/3})$) and subcritical for $r \geq 3$ (largest component ≤ 15 of 1,333 variables at $n = 4{,}000$), so the group sums factorize over bounded components; and this changes nothing about the theorem, because the group-enumeration term is about $1/256$ of the exponent's dominant term in the limit of the paper's parameters. A dense packed (Kronecker) representation of the measurement-indexed group sums is at best $1.23\times$ faster and at worst $57\times$ slower than a sparse convolution at the densities these designs produce, while the decomposition itself wins by up to $299.9\times$ over enumeration. (iii) Carried into practical counting, the same additive structure gives a component cache whose entries are histograms over a cardinality or pseudo-Boolean side constraint: it computes the entire histogram at 1.2–$1.3\times$ the cost of one plain count, whereas one CNF-encoded constraint value costs 0.6–$1{,}083\times$ the time of the entire histogram; this confirms with exact numbers what algebraic model counting predicts. Every result is reproducible from released code and a dataset with three independent verification runs.

1 Introduction

A model counter visits assignments; what it needs from an assignment is rarely the assignment itself. The counting algorithm of OpenAI [OpenAI, 2026b] that refutes the Exponential Time Hypothesis [Impagliazzo and Paturi, 2001, Impagliazzo et al., 2001] rests on a single device, made precise in its Definition 2.1: partition the variables into groups, assign each clause to one group, and let a clause depend on any other group only through a few bounded integer *measurements* of that group's assignment. Fixing the tuple of measurements separates the groups, and the count becomes a sum over measurement tuples of products of group sums (its identity (2.6)). The subexponential

running time comes from making the number of tuples and the cost per group both subexponential, at the price of an algebraic deformation whose constant term must be recovered by interpolation (its identities (2.9) and Section 2.5).

This paper takes that device as its subject and asks three questions that can be answered with exact computation at small scale: what the algorithm does, concretely, when every object in it is written down; what structure its own construction has beyond what its analysis uses; and whether the same device—indexing by an additive measurement—buys anything in the exact counting that is practised today. The answers are modest and, we think, useful; we state at the outset which of them are new.

Contributions. Each is labelled as **[new]**, **[reproduced]** (an independent reproduction of published work), **[known, confirmed]** (a known result confirmed with exact numbers), or **[correction]**.

1. **[reproduced]** An exact, executable model of the pipeline of OpenAI [2026b], Theorem 1.1: clause polynomials, replacement linear forms, measurement designs, the measurement-tuple identity (2.6), the restricted sums and inclusion–exclusion identity (2.9) over truncated power series, and Hermite interpolation of the deformed count H at $t = 1, \ldots, Q$. Three families of designs are implemented, including the paper's own Section 2.7 construction (auxiliary coordinates, swap matrices, interpolated f, eigenvector measurement rows, schedules and copies) at $r = 2$, $Q \in \{2, 3\}$, $R \in \{2, 4\}$ and $r = 3$. Every stage is checked against enumeration; $H(0) = \#F$ is recovered exactly through the full pipeline at $n \leq 14$ (Section 4).

2. **[new]** Under the paper's random schedules, the within-group clause hypergraph has exploration offspring mean $4/r^2$: critical at $r = 2$, where the largest within-group component is of order $n^{2/3}$ ($n = 1,000$: largest 83–109, $n^{2/3} = 100$; $n = 4,000$: largest 196–229, $n^{2/3} = 252$; $n = 16,000$: largest 685–1656, $n^{2/3} = 635$), and subcritical for $r \geq 3$, where it stays in the low double digits up to $n = 16{,}000$. The group sums of the algorithm therefore factorize over bounded components for $r \geq 3$. We also show that replacing the paper's group enumeration by a component product would change its running-time exponent by a fraction that is 14.6% at $\lambda = \lceil \log_2 n \rceil = 2^{26}$, the smallest power of two at which its parameters are defined and the dominant term is positive, and tends to about $1/256$, so the fact is structural, not algorithmic (Section 5, Appendix B).

3. **[new]** A dense, packed (Kronecker-substitution) representation of the measurement-indexed group sums, in which recombining two components is one big-integer multiplication, is at best 1.23× faster and at worst 57× slower than a sparse dictionary convolution on the 19 instances we can build, because only 4–24% of the measurement box is ever attained in the regime of few shared rows; the decomposition alone wins by up to 299.9× over enumeration (Section 6).

4. **[known, confirmed]** A component cache whose entries are histograms over an additive side measurement computes a cardinality or pseudo-Boolean histogram of a formula at 1.2–1.3× the cost of one plain weighted count, while one CNF-encoded cardinality value costs 0.6–1,083× the time of the whole histogram on the same engine, more than the histogram everywhere except at $b = 1$. This is algebraic model counting in the polynomial semiring [Kimmig et al., 2017] evaluated on the trace of a component-caching counter [Huang and Darwiche, 2005, 2007]; we confirm it with exact numbers, including grid totals that equal OEIS A006506 through 8×8 (Section 7).

5. **[correction]** The author's 2024–2025 manuscripts claimed that an oblivious interval sampler decides SAT in $O(n \log n)$ time. It does not: for every $n \geq 2$ there is a satisfiable formula of n unit clauses on which it answers UNSAT, and no procedure that reads the formula only through

evaluations can do better than 2^n queries deterministically or $2^n/3$ with bounded error. Section 3 states these facts as the zero-measurement end of the spectrum; the companion note [McElveen, 2026] gives the full treatment.

What this paper does not claim. It does not verify the theorem of OpenAI [2026b], whose formal verification is the Lean development released with it; a toy at $n \le 14$ certifies identities and constructions, not asymptotics. It does not exhibit a speed-up over any production model counter, and its practical section reproduces a known capability. It makes no claim about P versus NP. All timings are from a Python implementation on two virtual CPUs and are reported alongside deterministic operation counts.

Why executable models of AI-generated mathematics. The manuscript we study is one of 719 produced by an internal OpenAI model and released with Lean formalizations for about 42% of top-line results [OpenAI, 2026a]; the public discussion of that release has centred on the gap between machine-checked and humanly understood. A toy in which every measurement row, every rational input and every intermediate value can be printed and compared with brute force is a different kind of check from a proof assistant: it certifies less and explains more. We offer Section 4 in that spirit, as a comprehension aid that happens to be exact.

2 Preliminaries

Formulas and counting. A CNF formula F over variables $x_1, \dots, x_n$ is a set of clauses, each a disjunction of literals. $\#F$ is the number of satisfying assignments (models). #3-SAT is #P-complete [Valiant, 1979]. F is *bounded occurrence* if every variable occurs in at most three clauses; OpenAI [2026b] reduce general $\#k$-SAT to this form by counting-preserving sparsification [Impagliazzo et al., 2001, Calabro et al., 2006]. The Exponential Time Hypothesis [Impagliazzo and Paturi, 2001] asserts that 3-SAT has no $2^{o(n)}$-time algorithm. Write M for the number of literal occurrences and I_e for the variables of clause e.

The algorithm of OpenAI [2026b]. We keep that paper's notation and equation numbers. Each clause e with literals $L_{e,v}$ on $v \in I_e$ has the clause polynomial

$$\Phi_e(X) \;=\; 1 - \prod_{v \in I_e} \big(1 - L_{e,v}(X_v)\big), \qquad L_{e,v}(X_v) \in \{X_v,\ 1 - X_v\},$$

which is the satisfaction indicator on Boolean inputs, so $\#F = \sum_{x \in \{0,1\}^n} \prod_e \Phi_e(x)$. Each occurrence (e, v) is given a *replacement linear form* with polynomial coefficients, $\vec{P}_{e,v}(T) \in \mathbb{Q}[T]^n$, with $\langle \vec{P}_{e,v}(0), X \rangle = X_v$ and $\deg \vec{P}_{e,v} \le R$. The deformed count

$$H(T) \;=\; \sum_{x \in \{0,1\}^n} \prod_{e} \Phi_e\big(\langle \vec{P}_{e,v}(T), x \rangle_{v \in I_e}\big)$$

is a polynomial with $H(0) = \#F$ and $\deg H \le M \deg \vec{P}$. At each point $t \in [Q]$ an (r, τ, D)-*design* (Definition 2.1 of the paper) consists of groups $S_{t,1} \sqcup \cdots \sqcup S_{t,r} = [n]$ with $|S_{t,g}| \le 2n/r$, a clause assignment $\sigma_e(t) \in [r]$, and integer measurement matrices $C_{t,g}$ with at most $2\tau n$ rows in total and row absolute sums at most D, such that every form of a clause assigned to g depends on each other

group h only through $y_h = C_{t,h} x_h$:

$$\langle \vec{P}_{e,v}(t), x \rangle \;=\; \langle \vec{P}_{e,v}(t)|_{S_g}, x_g \rangle + \sum_{h \neq g} \langle \vec{\lambda}_{e,v,h}, y_h \rangle. \tag{2.5}$$

Given a design, $H(t)$ is computed by enumerating measurement tuples $\vec{y}$ and, for each, the group sums over the fibres $X_{t,g}(y_g) = \{u \in \{0,1\}^{S_g} : C_{t,g} u = y_g\}$:

$$H(t) \;=\; \sum_{\vec{y}} \prod_{g=1}^{r} \sum_{u \in X_{t,g}(y_g)} A_{t,g}(u; \vec{y}), \tag{2.6}$$

where $A_{t,g}(u;\vec{y})$ is the product of the clause polynomials of the clauses assigned to g, evaluated with u and the fixed measurements. Since Q values alone may not determine H, the paper computes $J = q + 1$ Taylor coefficients at each point, $q = \lfloor MR/Q \rfloor$, from restricted sums

$$Z_{t,E,U}(z) \;=\; \sum_{x : x_i = 1 \;\; (i \in E)} \prod_{e} \Phi_e\big(\langle \vec{P}_{e,v}(t), x \rangle + \langle \Delta \vec{P}_{e,v}(z), \mathbf{1}_U \rangle\big) \quad \text{in } \mathbb{Q}[z]/(z^J), \tag{2.8}$$

with $\Delta \vec{P}_{e,v}(z) = \vec{P}_{e,v}(t+z) - \vec{P}_{e,v}(t)$, through the inclusion–exclusion identity

$$H(t+z) \;\equiv\; \sum_{E \subseteq [n],\; |E| \leq q} \; \sum_{U \subseteq E} (-1)^{|E| - |U|} \, Z_{t,E,U}(z) \pmod{z^J}, \tag{2.9}$$

each $Z_{t,E,U}$ being computed by the algorithm of (2.6) over the truncated-series ring. The $QJ > \deg H$ coefficients determine H by a rational linear solve, and $H(0) = \#F$. The three costs multiply to, up to polynomial factors,

$$\sum_{j \leq \min(q,n)} \binom{n}{j} 2^j \;\cdot\; (2D+1)^{2\tau n} \;\cdot\; 2^{2n/r}, \tag{2.11}$$

and the parameter choices $r = \Theta(\sqrt{\log n}/(\log\log n)^2)$, $Q = \Theta(\log n / \log\log n)$, $R = \Theta(\sqrt{\log n})$, $\tau = \Theta(1/(r \log n))$, $D = r^{2Q}$ give $2^{O(n(\log\log n)^2/\sqrt{\log n})}$. The designs that achieve these parameters are built in the paper's Section 2.7; Section 4 reproduces that construction and Appendix A walks through it at $r = 2$, $Q = 2$, $R = 2$.

Component caching and semirings. Exact counters of the sharpSAT family [Bayardo and Pehoushek, 2000, Sang et al., 2004, Thurley, 2006, Korhonen and Järvisalo, 2021, Sharma et al., 2019] branch on a variable, propagate, split the residual formula into connected components, count each component recursively, and cache the count of each component under a canonical key. Huang and Darwiche [2005, 2007] showed that the trace of such a search is a decision-DNNF, so any query tractable on d-DNNF can be evaluated on it. Kimmig et al. [2017] formalized *algebraic model counting*: evaluating a d-DNNF with labels from a commutative semiring, of which weighted counting (the real semiring) and generating functions (the polynomial semiring) are instances. Section 7 uses exactly this: a component cache whose values live in the polynomial semiring of a side measurement.

3 The zero-measurement regime: oblivious sampling

The author's manuscripts [McElveen, 2024, 2025a,b] proposed to decide SAT by evaluating F on the assignments with indices $\{0, s, 2s, \dots\} \subset \{0, \dots, 2^n - 1\}$, $s = \lfloor 2^n/(c \log_2 n) \rfloor$, under the bijection between indices and assignments, and to answer UNSAT if none satisfies F. The set of evaluated assignments is fixed before the clauses are read; in the vocabulary of this paper it is the regime with no measurement at all. Two elementary facts settle it.

Theorem 3.1 (Incompleteness)**.** *For every $n \geq 2$ and every set $S \subsetneq \{0,1\}^n$ of assignments chosen without reading the formula, there is a satisfiable formula of n unit clauses, hence a satisfiable 3-CNF, with no model in S. For the sampler above with $c = 1$: at $n = 2$ the step is 4, only index 0 is evaluated, and $F = x_1 \wedge x_2$ is answered UNSAT; at $n = 3$ two of eight assignments are evaluated and $F = x_1 \wedge x_2 \wedge \neg x_3$ is answered UNSAT.*

Proof. Pick $b \notin S$ and let F_b be the conjunction of the n unit clauses forcing b; its unique model is b. □

Theorem 3.2 (Query lower bounds)**.** *Any deterministic procedure that reads F only through evaluations $F(x)$ makes 2^n evaluations on some satisfiable input before it can answer correctly; any randomized procedure with error at most $1/3$ makes at least $2^n/3$ evaluations in expectation on some input; any quantum procedure makes $\Omega(2^{n/2})$ queries, and Grover's algorithm achieves $O(2^{n/2})$.*

Proof. The deterministic bound is the adversary that answers 0 to every query until the last assignment. The randomized bound is Yao's principle [Yao, 1977] against the uniform distribution on the family $\{F_b\}$ together with the all-zero function: a procedure that queries fewer than $2^n/3$ points sees no 1 on a $2/3$ fraction of the F_b. The quantum bounds are Bennett et al. [1997] and Grover [1996]; see Buhrman and de Wolf [2002] for the survey. □

The sampler is *sound*—a reported assignment is a certificate—and its encodings (index bijections, factorial ranking of permutations, recoverability of sequences from pairwise sums) are correct but elementary. The companion note [McElveen, 2026] restates every claim of the source manuscripts as a theorem with proof or an explicit counterexample, shows why their reported benchmark outcomes cannot have been produced by the procedure as written, and measures the procedure against complete solvers. The positive lesson for the rest of this paper is only this: without a measurement of the assignment, no amount of structure in the *index* helps.

4 An executable model of the measurement-design pipeline

4.1 What is implemented

The module `measurement_toy.py` (standard library only; exact arithmetic with Python integers and `Fraction`s) implements, for a bounded-occurrence 3-CNF: the clause polynomials; replacement forms $\vec{P}_{e,v}(T)$ as sparse vectors of polynomials; designs as per-point objects (groups, clause assignment, integer rows); the measurement algorithm of (2.6) over any commutative ring with the fibres precomputed per group; the restricted sums of (2.8) over $\mathbb{Q}[z]/(z^J)$; the inclusion–exclusion (2.9); and Hermite interpolation of H from its Taylor data by exact Gaussian elimination. Condition (2.5) is not assumed: at every point, for every occurrence and every foreign group, the restriction of the form to that group is expressed in the row space of $C_{t,h}$ by a rational solve, and the run aborts if that fails. Three families of designs are provided.

Direct. Groups are fixed; each clause is assigned to the group holding most of its variables; $C_{t,h}$ has one unit row for each variable of S_h that occurs in a clause assigned elsewhere. The design holds at every t. With the own-group deformation $\vec{P}_{e,v} = e_v + Tw$ (w a random $\{-1, 0, 1\}$ vector on the assigned group), $\deg \vec{P} = 1$, H is a nonconstant polynomial of degree at most M, and the full pipeline runs with $q = \lfloor M/Q \rfloor = 1 < n/2$, the regime of the paper's Section 2.5. This design reads its interface directly and compresses nothing; it exercises the identities, not the saving.

Aux. The paper's own Section 2.7 construction at small parameters: auxiliary coordinates $B = ([r] \times [r])^Q$ with $D = r^{2Q}$; the swap matrices Π_s, $V_0 = \prod_s \Pi_s$, $W_0 = \frac{1}{2}\sum_s (I - \Pi_s)$; f of degree $\leq R - 1$ interpolating $(-1)^j$ on an interval I of R consecutive integers near Qp, $p = (r-1)/(2r)$; $M_0 = V_0 f(W_0)$; the eigenbasis of the swaps, with M_0 fixing every eigenvector whose count j of -1 positions lies in I and the transposes of the others forming the rows of C; the rows $\vec{P}_u(T)$ of (2.16) with the factors $\prod_{s \in E_{u,w}} (1 - T/s)$; schedules $a_v, d_e \in [r]^Q$ (a balanced greedy choice replaces the paper's derandomized balancing, whose hypothesis $12D^2 \leq M$ is false at these sizes); copies; the substitution (2.18); and the group-split measurement matrices $C_{t,g}$. The construction is checked as it is built: the action of M_0 on every eigenvector, the support condition $|E_{u,w}| \leq R - 1$ of (2.14), $\vec{P}_u(0) = e_u$, the degree bound, and that every row of $I - M_0$ lies in the row space of C. Groups, clause assignments and rows all change with t.

Collapse. One all-ones row per group at a single point, with $\vec{P} = e_v(1 - T) + s_h T$; it is kept to show why $Q = 1$ forces $q = M$.

4.2 The construction in the small

At $n = 12$ with $r = 2$, $Q = 2$, $R = 2$: $D = 16$ auxiliary coordinates, $f(j) = 1 - 2j$ interpolating $(-1)^j$ on $I = \{0, 1\}$, 15 of 16 eigenvectors fixed by M_0 and $c = 1$ measurement row (the eigenvector with $j = 2$, supported on four coordinates), 16 labels used by $M = 33$ occurrences with multiplicity $h = 3$, hence 3 copies. The designs at the two points, as printed from the dataset:

- $t = 1$: groups $\{2, 3, 7, 8, 9, 12\}, \{1, 4, 5, 6, 10, 11\}$; clause $\to$ group $(2, 1, 2, 1, 2, 1, 2, 2, 1, 1, 2)$; measurement rows $C_{1,1}$: $x_3 - x_{12}$, $x_7 - x_8$; $C_{1,2}$: $x_4 - x_{10}$, $x_1 - x_5$.
- $t = 2$: groups $\{2, 3, 5, 7, 10, 11\}, \{1, 4, 6, 8, 9, 12\}$; clause $\to$ group $(2, 2, 1, 1, 1, 1, 2, 2, 2, 1, 1)$; measurement rows $C_{2,1}$: $x_3 - x_{10}$, $-x_5 + x_7$; $C_{2,2}$: $x_4 - x_{12}$, $x_1 - x_8$.

The rows are shared ± 1 differences of two variables, exactly the shape the paper's measurements have; a clause assigned to group 2 reads a variable of group 1 only through them. The replacement form of the occurrence of x_{10} in clause 2 is $-\frac{1}{2}T\,x_3 - T\,x_4 + x_{10}$: it reads x_{10} at $T = 0$ and mixes in two other variables of the same group as T moves. The inputs at $t \neq 0$ are therefore rational, and so are the values: $H(1) = -12561/32$ and $H(2) = -9664$, each equal to the brute-force evaluation of $\sum_x \prod_e \Phi_e(\langle \vec{P}_{e,v}(t), x \rangle)$, while $H(0) = \#F = 736$.

4.3 Verification results

Table 1 lists every run of the toy in the dataset. The columns "(2.6) certified" and "$H(0)$" are the two checks: the measurement algorithm's value at every design point against brute force, and the interpolated constant term against the solution count. Independently of the toy, a second implementation (Section 9) expands $H(T)$ symbolically by brute force over all 2^n assignments and confirms every recorded value $H(t)$, every recorded Taylor coefficient, and $H(0)$, and re-tests the design conditions at every point with a fresh rank computation. In total 12 runs and 116 design points were certified.

The largest full-pipeline run is the direct design at $n = 14$ with $q = 1$: $Q = 20$ points, 580 restricted sums, 18,104,320 clause evaluations over $\mathbb{Q}[z]/(z^2)$, and $H(0) = 3108$ recovered exactly

Table 1: Runs of the executable model. M literal occurrences; Q points; q Taylor order per point; "$H(0)$" is the recovered constant term and the stage that produced it (Taylor = identity (2.9) with the stated number of restricted sums; values only = $Q > \deg H$ points interpolated directly). Wall time includes all brute-force checks.

n	m	M	design	Q	q	(2.6) certified	$\#F$	$H(0)$ (stage)	$= \#F$	s
8	7	21	direct, deformed	11	1	11 of 11	85	85 (Taylor, 187 restricted sums)	✓	3.6
10	9	27	direct, deformed	14	1	14 of 14	259	259 (Taylor, 294 restricted sums)	✓	7.9
12	11	33	direct, deformed	17	1	17 of 17	736	736 (Taylor, 425 restricted sums)	✓	30.8
14	13	39	direct, deformed	20	1	20 of 20	3108	3108 (Taylor, 580 restricted sums)	✓	552.8
14	13	39	direct, deformed, values only	40	0	40 of 40	3108	3108 (values only)	✓	62.5
12	11	33	direct, undeformed ($\deg P = 0$)	1	0	1 of 1	736	736 (constant H)	✓	0.1
6	5	15	aux (§2.7), r=2, Q=2, R=2	2	7	2 of 2	34	34 (Taylor, 1,458 restricted sums)	✓	1.4
12	11	33	aux (§2.7), r=2, Q=2, R=2	2	16	2 of 2	736	not run (1,062,882 sums)	–	2.8
12	11	33	aux (§2.7), r=2, Q=3, R=2	3	11	3 of 3	736	not run (1,582,035 sums)	–	5.4
12	11	33	aux (§2.7), r=3, Q=2, R=2	2	16	2 of 2	736	not run (1,062,882 sums)	–	2.6
8	7	21	aux (§2.7), r=2, Q=3, R=4=Q+1	3	21	3 of 3	85	not run (19,683 sums)	–	0.1
7	7	21	collapse	1	21	1 of 1	48	not run (2,187 sums)	–	0.0

Table 2: The three factors of (2.11) at toy scale: pairs (E, U) per point, measurement tuples attained (and the paper's box bound $\prod_g (2D+1)^{c_g}$ at $t = 1$), assignments within groups $\sum_g 2^{|S_g|}$, against 2^n.

n	design	Taylor pairs/pt	tuples	box bound	group assignments	2^n
8	direct, deformed	17	64	729	32	256
10	direct, deformed	21	32	243	64	1,024
12	direct, deformed	25	32	243	128	4,096
14	direct, deformed	29	1,024	59,049	256	16,384
14	direct, deformed, values only	1	1,024	59,049	256	16,384
12	direct, undeformed ($\deg P = 0$)	1	32	243	128	4,096
6	aux (§2.7), r=2, Q=2, R=2	729	12	75	20	64
12	aux (§2.7), r=2, Q=2, R=2	531,441	81	625	128	4,096
12	aux (§2.7), r=2, Q=3, R=2	527,345	1,024	158,203,125	272	4,096
12	aux (§2.7), r=3, Q=2, R=2	531,441	1,024	703,125	80	4,096
8	aux (§2.7), r=2, Q=3, R=4=Q+1	6,561	1	1	32	256
7	collapse	2,187	20	63	24	128

from 40 Hermite conditions in 9.2 minutes. Through the paper's own construction the full pipeline is run at $n = 6$ (where $Q = 2$ gives $q = \lfloor M/Q \rfloor = 7 \geq n$ with $M = 15$, so the Taylor stage is exhaustive: 1,458 restricted sums) and identity (2.6) is certified at every point at $n = 12$ for $(r, Q, R) = (2, 2, 2), (2, 3, 2), (3, 2, 2)$, and at $n = 8$ for $(2, 3, 4)$.

4.4 The three costs at toy scale

Table 2 prints the three factors of (2.11) for every run. Nothing beats 2^n, and nothing should: the saving needs $R \ll Q$ with $r^{2Q} \ll n$, which the paper's parameters place at n beyond any computation, and its own balancing condition $12D^2 \leq M$ already asks for 3,072 occurrences at $D = 16$ where the toy has 33. The case $R = Q + 1$ is instructive in the other direction: f then fixes every eigenvector, $c = 0$, no measurements exist, and every form reads its variable from the assigned group at every point at the price of degree Q—the paper's remark that "choosing a different linear form at each point and interpolating would generally cost degree Q" made concrete.

Table 3: Largest within-group component (variables) under uniform schedules at one point; ranges over three seeds. $3/r$ is the expected within-group degree of a variable, $4/r^2$ the offspring mean of the exploration.

r	$3/r$	$4/r^2$	$n = 1,000$ (group $1,000/r$)	$n = 4,000$ (group $4,000/r$)	$n = 16,000$ (group $16,000/r$)
2	1.50	1.00	83–109	196–229	685–1656
3	1.00	0.44	10–12	12–15	15–19
4	0.75	0.25	5–8	8–12	10–14
5	0.60	0.16	4–7	6–7	7–11
6	0.50	0.11	3–4	5	6–7
8	0.38	0.06	3	4	4–5
12	0.25	0.03	2–3	2–3	3–4

5 Component structure of the construction's groups

5.1 A branching-process heuristic and its confirmation

In the paper's Step 4 every variable v receives a schedule $a_v \in [r]^Q$ and every clause e a schedule $d_e \in [r]^Q$; at point t, v belongs to group $a_v(t)$ and e is assigned to group $d_e(t)$. The paper's Lemma 4.5 derandomizes a uniform random choice of schedules subject to balance constraints; we study the uniform random choice itself, which is the model the lemma derandomizes. Fix t and consider the *within-group clause hypergraph*: a clause assigned to g couples those of its three variables that lie in $S_{t,g}$. Only this hypergraph couples the variables of a group inside the group sum of (2.6); every other dependence enters through the fixed measurements.

Proposition 5.1. *Under uniform independent schedules, explore the within-group hypergraph from a variable of group g. Each of the variable's three clauses is assigned to g with probability $1/r$, and each of the other two variables of such a clause lies in g with probability $1/r$; a variable reached through a clause has two further clauses. The offspring mean of a non-root variable is therefore $\mu = 2 \cdot (1/r) \cdot 2 \cdot (1/r) = 4/r^2$: the exploration is critical at $r = 2$ and subcritical for $r \geq 3$. In the branching-process approximation [Athreya and Ney, 1972] the largest component is of order $n^{2/3}$ at $r = 2$ and $O(\log n)$ for $r \geq 3$.*

The proposition is a heuristic in the usual sense (the exploration is not exactly a branching process because variables and clauses are sampled without replacement), so we test it. Table 3 gives the largest within-group component for random bounded-occurrence 3-CNFs with $m = n$ clauses under uniform schedules at one point, three seeds each; Figure 1 plots it against n. At $r = 2$ the largest component is of order $n^{2/3}$, with the wide fluctuations typical of criticality ($n = 1,000$: largest 83–109, $n^{2/3} = 100$; $n = 4,000$: largest 196–229, $n^{2/3} = 252$; $n = 16,000$: largest 685–1656, $n^{2/3} = 635$); for $r \geq 3$ it stays in single or low double digits up to $n = 16{,}000$, growing at most logarithmically. An independent breadth-first recomputation reproduces every entry (Section 9).

5.2 What it does and does not imply for the algorithm

For $r \geq 3$ the group sum $\sum_{u \in X_{t,g}(y_g)} A_{t,g}(u; \vec{y})$ factorizes: the measurement $y_g = C_{t,g}u$ is additive across the components of the within-group hypergraph, so the histogram of the group over its measurement box is the convolution of the component histograms, each computable by enumerating a bounded number of assignments. The paper's Step 2, which enumerates all $2^{|S_{t,g}|} \leq 2^{2n/r}$ assignments of a group, could be replaced by a convolution whose cost is governed by the number of lanes of the box. Since the paper takes $r \to \infty$, its own regime is deep inside the subcritical range.

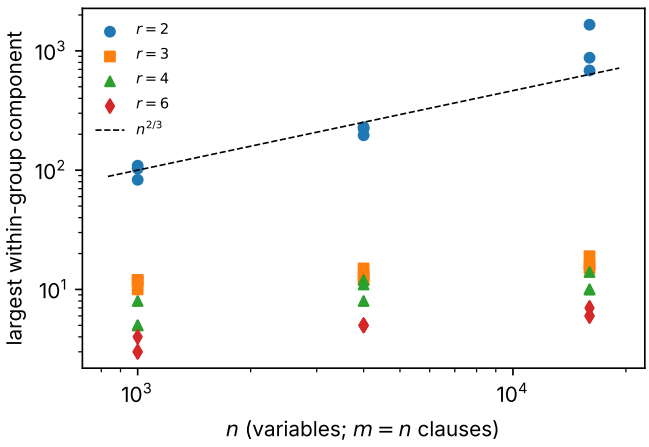


Figure 1: Largest within-group component against n (three seeds per point). At $r = 2$ it is of order $n^{2/3}$; for $r \geq 3$ it grows at most logarithmically.

This does not improve the theorem. Taking logarithms of the three factors of (2.11) gives the exponent

$$n\Big[\underbrace{\tfrac{R}{Q}\log_2\tfrac{2Q}{R}}_{T_1\ \text{(Taylor data)}} + \underbrace{\tau\log_2(2D+1)}_{T_2\ \text{(measurements)}} + \underbrace{\tfrac{1}{r}}_{T_3\ \text{(group)}}\Big]$$

up to the paper's constants, and under its parameter choices (2.19)—$\lambda = \lceil \log_2 n \rceil$, $\nu = \lceil \log_2 \lambda \rceil$, $r = \lfloor \sqrt{\lambda}/\nu^2 \rfloor$, $Q = \lfloor \lambda/(128\nu) \rfloor$, $R = 4\lceil \sqrt{\lambda} \rceil$, $\tau = 1/(r\lambda)$, $D = r^{2Q}$—one has, to leading order, $T_1 \approx 256\,\nu^2/\sqrt{\lambda}$, $T_3 \approx \nu^2/\sqrt{\lambda}$ and $T_2 \approx \nu^2/(128\sqrt{\lambda})$ (Appendix B). Removing T_3 entirely changes the exponent by about one part in 256 in the limit; at finite n its share is larger (14.6% at $\lambda = 2^{26}$, the smallest power of two at which every parameter is defined and $T_1 > 0$; 2.0% at 2^{32}; 0.69% at 2^{64}; Appendix B). The component structure is a fact about the construction, not a route to a faster algorithm; the dominant cost is the Taylor data, $\sum_{j \leq q} \binom{n}{j} 2^j$ restricted sums per point, and it is untouched.

6 Dense versus sparse recombination of group sums

The inner sum of (2.6) for one group and one foreign tuple is a histogram: the weighted number of assignments u of the group at each attained measurement vector y_g. If the group splits into components, the histogram is the convolution of the component histograms. Two representations compete. A *sparse dictionary* stores attained vectors only and convolves pairwise. A *dense packed* representation lays the whole measurement box out as lanes of one big integer (Kronecker substitution [Harvey, 2009]: lane index $\sum_i (y_i + D_i^-) \prod_{j<i} (2D_j + 1)$, lane width W bits), so that convolving two histograms is one big-integer multiplication and the lanes add exactly as measurements add under concatenation of partial assignments. The packed form is the natural home of the "additive homomorphism" in the paper's device; whether it pays is an empirical question about density.

6.1 Method

`fibre_loop.py` computes the histogram of one group of k variables three ways—*bucket* (enumerate 2^k assignments, bucket by y), *dict* (components, sparse convolution), *packed* (components, Kronecker

Table 4: The fibre loop three ways (ms). "comp." is the largest component; "enumerated" the assignments visited by the component methods against 2^k; "attained" the nonzero lanes of the box (and their fraction of it). Ratios are bucket/packed and dict/packed.

k	rows	lanes	comp.	enumerated	attained	bucket	dict	packed	b/p	d/p
Real Section 2.7 design, $r = 2$, $t = 1$										
8	3	27	8	256	10 (37%)	2.6	0.9	2.9	0.91	0.31
10	4	54	9	514	30 (56%)	5.2	3.7	5.4	0.97	0.68
12	5	162	12	4,096	126 (78%)	18.7	18.7	15.5	1.21	1.21
14	5	243	13	8,194	243 (100%)	76.4	30.8	27.8	2.75	1.11
16	6	729	16	65,536	210 (29%)	242	226	192	1.26	1.18
18	7	2,187	15	32,776	810 (37%)	1,047	137	135	7.73	1.01
20	7	1,458	19	524,290	200 (14%)	4,138	2,801	2,281	1.81	1.23
Synthetic measurement, $r = 4$, 8 rows										
16	8	65,536	2	32	5,248 (8%)	264	48.2	689	0.38	0.07
18	8	65,536	2	36	2,832 (4%)	864	12.6	398	2.17	0.03
20	8	65,536	2	40	5,876 (9%)	3,860	37.8	552	6.99	0.07
22	8	65,536	2	44	15,616 (24%)	14,384	150	879	16.37	0.17
24	8	65,536	2	48	14,720 (22%)	67,882	–	655	103.57	–
Synthetic measurement, $r = 3$, 8 rows										
16	8	65,536	2	32	7,328 (11%)	250	40.4	306	0.82	0.13
18	8	65,536	3	42	2,560 (4%)	1,290	14.9	329	3.92	0.05
20	8	65,536	2	40	8,240 (13%)	3,707	63.7	430	8.63	0.15
22	8	65,536	6	104	10,784 (16%)	17,776	59.3	505	35.20	0.12
Synthetic measurement, $r = 4$, 10 rows										
20	10	1,048,576	2	40	39,528 (4%)	3,892	180	10,347	0.38	0.02
22	10	1,048,576	2	44	96,224 (9%)	15,959	1,095	9,615	1.66	0.11
24	10	1,048,576	2	48	53,888 (5%)	65,924	309	9,672	6.82	0.03

lanes, GMP multiplication, lanes reduced modulo p after every product)—on one shared vectorized kernel for enumeration and clause evaluation, so that only the algorithm differs, with all arithmetic exact modulo $p = 2^{31} - 1$ (a production implementation would use several primes and the Chinese remainder theorem; the cost per prime is what is measured). Two instance families are used. The *real design*: the group, forms, shifts and rows of the Section 2.7 construction at $r = 2$, $t = 1$, for a foreign tuple attained by a random assignment. A *synthetic measurement* for $r \geq 3$: the within-group clause structure of a random bounded-occurrence 3-CNF under uniform schedules, with Boolean own inputs, fixed foreign inputs, and c sparse ± 1 rows of support three per group. The synthetic family is needed because at these sizes the real construction either couples the group into one component ($r = 2$) or, for $r \geq 3$ at $Q = 2$, produces 9–10 rows per copy and a box larger than 2^k; the paper's regime of few shared rows exists only at enormous n. Every histogram is compared across the three methods; 3 of them, with $k \leq 12$, are also compared with the exact rational fibre sums of the toy, and every one with $k \leq 16$ is recomputed by plain enumeration in the independent check.

6.2 Results

Table 4 and Figure 2 give the numbers. Three readings. First, decomposition is the entire gain: against the 2^k enumeration the better of the two component methods wins by 14.6–299.9× for $k \geq 20$ at $r \geq 3$, and by at most 7.7× at $r = 2$, where a win appears only when a small piece splits off the giant component. Second, the packed representation adds nothing over the sparse one: the ratio of dictionary time to packed time is 0.02–1.23 across the 19 instances, so the packed form is at best 1.23× faster (on 5 small real-design groups) and at worst 57× slower, because the dense form pays for every lane while only 4–24% are attained in the synthetic family (Table 4), and

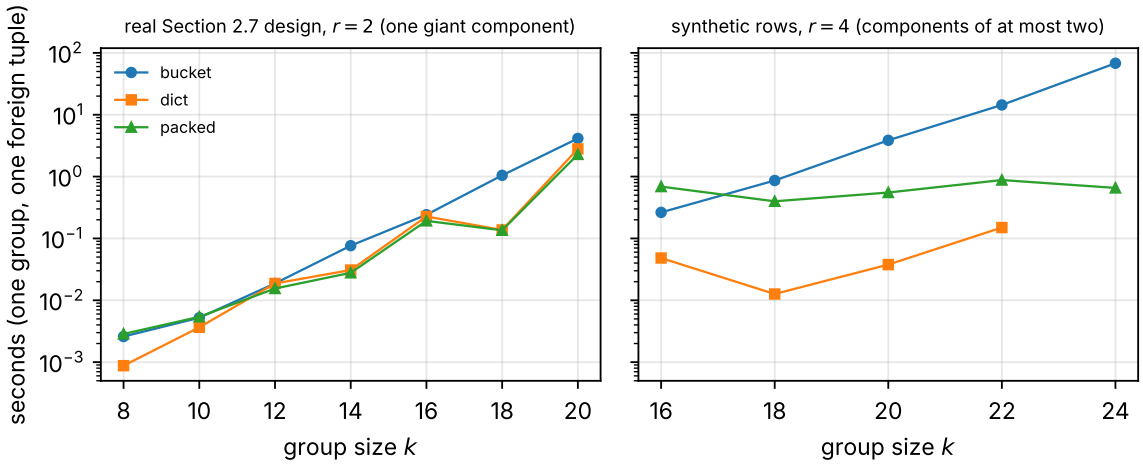


Figure 2: Time of the three methods against group size. Left: the real design at $r = 2$, one giant component, where decomposition rarely helps. Right: the synthetic measurement at $r = 4$, components of at most two variables, where enumeration grows as 2^k and both component methods stay nearly flat.

the fraction falls as rows are added, the box being a product of independent ranges in which the attained measurements concentrate near zero. Writing both sides in C would speed them up by similar factors; the ratio is governed by density. Third, the one place a dense form would win is a dense histogram—a single low-range row, such as a cardinality constraint—which is the subject of the next section, and which is served well by the sparse form anyway.

7 Semiring component caches under side constraints

7.1 The engine

`constrained_count.py` is a miniature sharpSAT-style counter (unit propagation; decomposition of the residual formula into connected components; a cache keyed by the canonical residual component; static or dynamic branching) whose cached values are histograms. Given literal weights $w_0[v], w_1[v]$ and c integer rows $a^{(1)}, \ldots, a^{(c)}$, it computes

$$\mathcal{H}(y) \;=\; \sum_{x \models F,\; Ax = y} \; \prod_{v} w_{x_v}[v] \qquad \text{for every attained } y \in \mathbb{Z}^c,$$

by letting a forced literal contribute $w_{x_v}[v]\, z^{x_v a_v}$, a free variable contribute $w_0[v] + w_1[v] z^{a_v}$, and components combine by convolution. With $c = 1$ and $a = (1, \ldots, 1)$ this is the weighted count by cardinality; $c = 0$ is plain weighted counting on the same code. By Huang and Darwiche [2005] the search is a decision-DNNF and by Kimmig et al. [2017] this is its evaluation in the polynomial semiring; nothing in the engine is new, which is the point of the comparison.

The comparison arm is the standard route for an ordinary counter: a functional, counting-preserving sequential-counter encoding of $\sum_v x_v = b$ [after Sinz, 2005], with auxiliaries $s(i, j) \leftrightarrow$ (at least j of $x_1, \ldots, x_i$) for $1 \le j \le \min(i, b + 1)$, appended to F and counted by the same engine with $c = 0$. Propagation, decomposition, cache and branching are shared; only the treatment of the constraint differs.

Table 5: Independent sets of the $W \times H$ grid by size (static branching order). One histogram run yields every size; each encoding run yields one size b. "comp." is the largest component met; the parenthesized ratio is the encoding's time over the whole histogram's.

grid	n	sizes	ms	nodes	cache/hits	total	encoding, one b
4×4	16	9	3.0	43	43/33	1,234 = OEIS	b=1: 31 aux, 3.4 ms, 15 nodes, comp. 31 (1.1×); b=2: 45 aux, 13.2 ms, 50 nodes, comp. 46 (4.4×); b=4: 70 aux, 38.2 ms, 95 nodes, comp. 73 (12.8×)
5×5	25	14	13.9	111	111/101	55,447 = OEIS	b=1: 49 aux, 10.7 ms, 24 nodes, comp. 49 (0.8×); b=3: 94 aux, 114 ms, 201 nodes, comp. 96 (8.2×); b=6: 154 aux, 430 ms, 379 nodes, comp. 159 (31.0×)
6×6	36	19	37.9	262	262/257	5,598,861 = OEIS	b=1: 71 aux, 22.6 ms, 35 nodes, comp. 71 (0.6×); b=4: 170 aux, 1,096 ms, 655 nodes, comp. 173 (28.9×); b=9: 315 aux, 5,068 ms, 1,344 nodes, comp. 323 (133.7×)
7×7	49	26	278	583	583/591	1,280,128,950 = OEIS	–
8×8	64	33	608	1,246	1,246/1,280	660,647,962,955 = OEIS	–

Table 6: Random 3-CNF (dynamic branching order): the cardinality histogram against the plain weighted count on the same engine, and against the encoding of one b.

n	m	weights	entries	hist. ms	nodes	plain ms	hist/plain	encoding, one b
20	50	unweighted	11	5.2	78	4.3	1.22	b=5: 105 aux, 27.3 ms, 16 nodes, comp. 109 (5.2×); b=10: 165 aux, 207 ms, 212 nodes, comp. 174 (39.8×)
30	75	weights $w_1 \in \{1, 2, 3\}$	21	44.5	557	36.0	1.24	b=7: 212 aux, 343 ms, 252 nodes, comp. 218 (7.7×); b=15: 360 aux, 17,968 ms, 7,644 nodes, comp. 374 (403.5×)
40	100	weights $w_1 \in \{1, 2, 3\}$	26	329	2,186	261	1.26	b=10: 385 aux, 4,249 ms, 658 nodes, comp. 394 (12.9×); b=20: 630 aux, 356,479 ms, 97,698 nodes, comp. 649 (1083.4×)

7.2 Results

Tables 5 and 6 and Figure 3 give the numbers; every grid total equals OEIS A006506 [OEIS Foundation Inc., 2026, Calkin and Wilf, 1998], every histogram with $n \leq 20$ equals brute force, and every encoded count equals its histogram entry. Three readings. The histogram-valued cache computes the entire histogram at 1.2–1.3× the cost of one plain count: the search and the cache are identical and only the stored values change; on the 6×6 grid a pseudo-Boolean row with $a_v \in \{1, 2, 3\}$ (37 values) costs 43.0 ms and two rows (left-half and right-half counts, 100 pairs) 50.8 ms against 37.9 ms for the cardinality row. The encoding route multiplies the cost of a single b by up to 134× on grids and 1,083× on random 3-CNF, growing with b and n, because the counter chain couples every variable: the largest component grows from n to n plus the auxiliaries (36 to 323 on the 6×6 grid at the mode, where the ratio is 134×; 40 to 649 at $n = 40$, $b = 20$, where one encoded value takes 356 s and 97,698 nodes against 329 ms for all 26 values, a ratio of 1,083×), decomposition disappears, and the cache fills with states that differ only in their prefix count. This is the known weakness of CNF-encoded cardinality constraints for counters and the known remedy; for grid independent sets the remedy reduces to the transfer-matrix method of Calkin and Wilf [1998], which is also what our independent check uses. The production compilers [Lagniez and Marquis, 2017] produce d-DNNF on which the same semiring evaluation is a post-processing step; what we have not measured, for want of those tools in our environment, is the constant-factor comparison between a top-down histogram cache and compilation followed by evaluation.

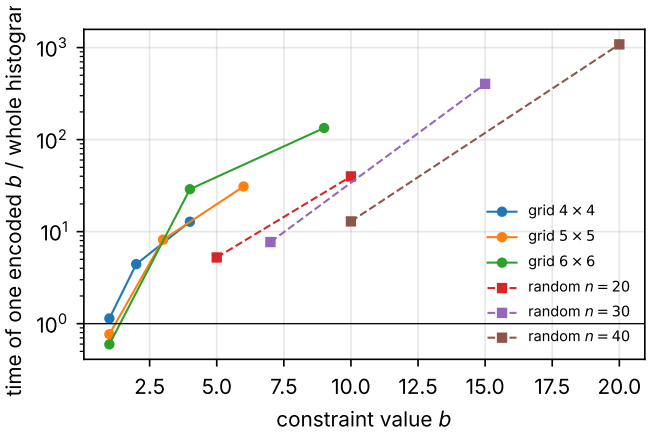


Figure 3: Cost of one CNF-encoded cardinality value b relative to the whole histogram computed by the semiring cache, on the same engine.

8 Discussion

What an additive measurement buys. The three regimes line up on one axis. With no measurement, the oblivious sampler of Section 3 cannot see the formula and fails on a forced assignment. With lossy, bounded, algebraically deformed measurements, the algorithm of OpenAI [2026b] separates groups at the cost of an interpolation whose Taylor-data term dominates its exponent; its construction has more structure than its analysis uses (Section 5), and the natural dense representation of its group sums buys nothing at reachable densities (Section 6). With an exact additive side measurement carried through a component cache, practical constrained counting gets its histogram nearly for free (Section 7); that is the generating-function structure of algebraic model counting, and it was known. The homomorphism that pays, in the end, is the additivity of the side constraint across disjoint components, in sparse form.

Verification and comprehension of AI-generated proofs. The Lean development accompanying OpenAI [2026b] certifies the top-line statement with far more authority than any computation at $n \leq 14$. What the executable model adds is of a different kind: a reader can print the design at $t = 1$ and $t = 2$ (Section 4.2), see that a measurement is $x_3 - x_{12}$, see an input become $-\frac{1}{2}Tx_3$, and see $H(1) = -12561/32$ agree with a brute-force sum. We suggest that releases of machine-generated mathematics would benefit from such models alongside their formalizations, and we release ours as a contribution to the community revision of this manuscript [OpenAI, 2026a].

Limitations. The toy certifies identities and constructions at sizes where every quantity is enumerable; it says nothing about asymptotics. The greedy schedules of the aux design replace the paper's derandomized balancing, whose hypothesis fails at toy sizes. The synthetic measurement of Section 6 is a stand-in for a regime the real construction reaches only at astronomical n. The counter of Section 7 is a miniature in Python; its absolute times are not those of a production tool, and the ratios it measures are between two arms of the same engine. All wall-clock times are single runs on shared virtual hardware; the deterministic counts beside them (nodes, assignments, lanes, cache entries) carry the argument.

Future work (speculative, not a result). The author's prior work on computing on compressed data suggests asking whether a component cache can hold its histograms in a compressed representation that still admits convolution without decompression. Section 6 shows that dense lane packing is not that representation at the densities seen here; a sparse, additively homomorphic encoding of attained measurement vectors would have to beat a hash map on both time and memory to matter, and we know of no evidence that it does. We record the question and no claim.

9 Reproducibility

Code and dataset. The repository, `https://github.com/chelsinchaos/measurement-indexed-counting`, contains `measurement_toy.py` (the executable model; standard library only), `fibre_loop.py` (Section 6; NumPy and gmpy2), `constrained_count.py` (Section 7; standard library only), `run_all.py` (regenerates the dataset), `independent_check.py` (verification run 2), and `make_tables.py` (produces every table, figure and number in this paper from the dataset). The dataset holds every formula in DIMACS with its generator and seed, every design of the toy as JSON, the results of the four experiments as JSON with environment metadata, the logs, and a manifest with the SHA-256 of each of its 64 files. Environment: Python 3.13.18, NumPy 2.5.3, gmpy2 2.3.2, 2 virtual CPUs, Linux-6.18.44-fc-v180-x86_64-with-glibc2.39.

Three verification runs. *Run 1* is the generation itself, which aborts on any failed internal check: the measurement algorithm against brute force at every design point, $H(0)$ against the solution counter, the three fibre-loop methods against each other and against exact rational fibre sums, grid totals against OEIS A006506, histograms against brute force at $n \leq 20$, and every encoded count against its histogram entry. *Run 2* (`independent_check.py`) recomputes the results by different algorithms written separately from the generating code: the deformed count $H(T)$ as an explicit polynomial by symbolic brute force over all assignments (which also confirms every recorded Taylor coefficient and re-tests the design conditions with a fresh rank routine); component sizes by breadth-first search instead of union-find; fibre histograms with $k \leq 16$ by plain enumeration from the stored clauses, forms, shifts and rows; grid histograms by a transfer-matrix dynamic programme that never decomposes or caches; and random-3-CNF histograms by exhaustive DPLL enumeration that visits every model. It passed 74 of 74 checks. *Run 3* is an audit by an agent with no access to the development history. It re-executed the generation in quick mode in a clean directory (quick mode skips the four longest runs: the toy at $n = 14$ with $q = 1$, the component statistics at $n = 16{,}000$, the two $k = 24$ fibre instances and the encoding at $n = 40$, $b = 20$; the first and last of these are covered by Run 2's symbolic expansion and exhaustive enumeration, while the $n = 16{,}000$ statistics and the $k = 24$ timings rest on the single generation run), found zero differences in deterministic quantities, re-ran the independent check (74 of 74), recomputed 45 of this paper's generated numbers from the raw JSON with its own code, and found six defects in the prose of the first draft, all corrected in this version. Its report is `VERIFICATION.md` in the repository.

Acknowledgements

The implementation, experiments and drafting were carried out with Claude (Anthropic) as an AI research assistant under the author's direction; the author is responsible for all content and for any errors. The manuscript studied here was made available by OpenAI under the Apache-2.0 licence of its repository.

References

Krishna B. Athreya and Peter E. Ney. *Branching Processes*. Springer, 1972.

Roberto J. Bayardo and Joseph Daniel Pehoushek. Counting models using connected components. In *Proc. 17th National Conference on Artificial Intelligence (AAAI)*, pages 157–162, 2000.

Charles H. Bennett, Ethan Bernstein, Gilles Brassard, and Umesh Vazirani. Strengths and weaknesses of quantum computing. *SIAM Journal on Computing*, 26(5):1510–1523, 1997.

Harry Buhrman and Ronald de Wolf. Complexity measures and decision tree complexity: a survey. *Theoretical Computer Science*, 288(1):21–43, 2002.

Chris Calabro, Russell Impagliazzo, and Ramamohan Paturi. A duality between clause width and clause density for SAT. In *Proc. 21st IEEE Conference on Computational Complexity (CCC)*, pages 252–260, 2006.

Neil J. Calkin and Herbert S. Wilf. The number of independent sets in a grid graph. *SIAM Journal on Discrete Mathematics*, 11(1):54–60, 1998.

Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proc. 28th ACM Symposium on Theory of Computing (STOC)*, pages 212–219, 1996.

David Harvey. Faster polynomial multiplication via multipoint Kronecker substitution. *Journal of Symbolic Computation*, 44(10):1502–1510, 2009.

Jinbo Huang and Adnan Darwiche. DPLL with a trace: From SAT to knowledge compilation. In *Proc. 19th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 156–162, 2005.

Jinbo Huang and Adnan Darwiche. The language of search. *Journal of Artificial Intelligence Research*, 29:191–219, 2007.

Russell Impagliazzo and Ramamohan Paturi. On the complexity of k-SAT. *Journal of Computer and System Sciences*, 62(2):367–375, 2001.

Russell Impagliazzo, Ramamohan Paturi, and Francis Zane. Which problems have strongly exponential complexity? *Journal of Computer and System Sciences*, 63(4):512–530, 2001.

Angelika Kimmig, Guy Van den Broeck, and Luc De Raedt. Algebraic model counting. *Journal of Applied Logic*, 22:46–62, 2017.

Tuukka Korhonen and Matti Järvisalo. Integrating tree decompositions into decision heuristics of propositional model counters. In *Proc. 27th International Conference on Principles and Practice of Constraint Programming (CP)*, volume 210 of *LIPIcs*, pages 8:1–8:11, 2021.

Jean-Marie Lagniez and Pierre Marquis. An improved decision-DNNF compiler. In *Proc. 26th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 667–673, 2017.

Chelsea A. McElveen. Efficient permutational algorithms for NP-complete problems. Manuscript, 1 December 2024, 2024.

Chelsea A. McElveen. Formal symbolic logic proof that P = NP via deterministic bitstring traversal SAT solver. Manuscript, March 2025, 2025a.

Chelsea A. McElveen. Deterministic polynomial-time algorithms for NP-complete problems provide a constructive proof that P = NP. Manuscript, 2025, 2025b.

Chelsea A. McElveen. Oblivious sampling cannot decide SAT: a correction to the author's 2024–2025 claims. Companion note, 2026, 2026.

OEIS Foundation Inc. The on-line encyclopedia of integer sequences, sequence A006506. `https://oeis.org/A006506`, 2026.

OpenAI. openai/math: mathematical manuscripts and proof artifacts produced by an internal OpenAI model. `https://github.com/openai/math`; community revision fork `https://github.com/mathcommunity/oai-math-revised`, 2026a.

OpenAI. The Exponential Time Hypothesis is false. Manuscript produced by an internal OpenAI model, family 378 ("Disproof of the Exponential Time Hypothesis") of `https://github.com/openai/math`, folder `preprints/The-Exponential-Time-Hypothesis-is-False-October-25-2026`, with a Lean formalization of the top-line result (`lean/docs/378.md`), 2026b. Dated 17 October 2026, revised 25 October 2026; 67 pp.

Tian Sang, Fahiem Bacchus, Paul Beame, Henry Kautz, and Toniann Pitassi. Combining component caching and clause learning for effective model counting. In *Proc. 7th International Conference on Theory and Applications of Satisfiability Testing (SAT)*, 2004.

Shubham Sharma, Subhajit Roy, Mate Soos, and Kuldeep S. Meel. GANAK: A scalable probabilistic exact model counter. In *Proc. 28th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 1169–1176, 2019.

Carsten Sinz. Towards an optimal CNF encoding of Boolean cardinality constraints. In *Proc. 11th International Conference on Principles and Practice of Constraint Programming (CP)*, volume 3709 of *LNCS*, pages 827–831, 2005.

Marc Thurley. sharpSAT – counting models with advanced component caching and implicit BCP. In *Proc. 9th International Conference on Theory and Applications of Satisfiability Testing (SAT)*, volume 4121 of *LNCS*, pages 424–429, 2006.

Leslie G. Valiant. The complexity of enumeration and reliability problems. *SIAM Journal on Computing*, 8(3):410–421, 1979.

Andrew Chi-Chih Yao. Probabilistic computations: Toward a unified measure of complexity. In *Proc. 18th IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 222–227, 1977.

A The Section 2.7 construction, step by step at $r = 2$, $Q = 2$, $R = 2$

We record the construction as implemented, with the quantities that the reader can recompute by hand. Groups are $\{1, 2\}$ and points are $t \in \{1, 2\}$.

Step 1 (coordinates). $B = (\{1, 2\} \times \{1, 2\})^2$, $D = 16$. An index $u = ((a_u(1), d_u(1)), (a_u(2), d_u(2)))$ records, at each point, the group $a_u(t)$ of the coordinate and the group $d_u(t)$ of the clause that will read it. The swap Π_s exchanges the two entries of pair s.

Step 2 (the constant matrix). $p = (r-1)/(2r) = 1/4$, $Qp = 1/2$, $A_0 = \lfloor Qp - (R-1)/2 \rfloor = 0$, $I = \{0,1\}$, and f is the line through $(0,1)$ and $(1,-1)$: $f(j) = 1-2j$. With $W_0 = \frac{1}{2}\sum_s (I - \Pi_s)$ and $V_0 = \Pi_1\Pi_2$,

$$M_0 \;=\; V_0 f(W_0) \;=\; \Pi_1\Pi_2\big(-I + \Pi_1 + \Pi_2\big) \;=\; \Pi_1 + \Pi_2 - \Pi_1\Pi_2.$$

Per position the swap has eigenvectors $e_{11}, e_{22}, e_{12}+e_{21}$ (eigenvalue $+1$) and $e_{12}-e_{21}$ (eigenvalue -1); the product basis of B has $j \in \{0,1,2\}$ positions with eigenvalue -1, and $M_0\psi = (-1)^j f(j)\psi$, which equals ψ for $j \in I$ and -3ψ for $j = 2$. Fifteen of the sixteen basis vectors are fixed; the remaining one, $(e_{12}-e_{21}) \otimes (e_{12}-e_{21})$, is the single row of C, supported on the four coordinates with both pairs in $\{(1,2),(2,1)\}$.

Step 3 (the rows). For u and a nonzero $M_0[u,w]$, $E_{u,w} = \{t : a_w(t) \neq d_u(t)\}$; for $w = \Pi_1 u$ this is empty at $t = 1$ and contains $t = 2$ exactly when $a_u(2) \neq d_u(2)$, so $|E_{u,w}| \leq 1 = R-1$. Then

$$\langle \vec{P}_u(T), \zeta \rangle \;=\; \big((I - M_0)\zeta\big)_u + \sum_w M_0[u,w] \prod_{s \in E_{u,w}} \Big(1 - \frac{T}{s}\Big)\,\zeta_w,$$

which equals ζ_u at $T = 0$; at $t = 1$ the surviving ζ_w have $a_w(1) = d_u(1)$, i.e. lie in the assigned group, and the first term is a multiple of the measurement $C\zeta$. For $u = ((1,2),(1,2))$ one obtains $\langle \vec{P}_u(T), \zeta \rangle = [\zeta_u - \zeta_{\Pi_1 u} - \zeta_{\Pi_2 u} + \zeta_{\Pi_1\Pi_2 u}] + (1 - T/2)\zeta_{\Pi_1 u} + (1-T)\zeta_{\Pi_2 u} - \zeta_{\Pi_1\Pi_2 u}$: collecting coefficients, the row is $\zeta_u - \frac{T}{2}\zeta_{\Pi_1 u} - T\zeta_{\Pi_2 u}$, which after the substitution of Step 4 is exactly the example form $-\frac{1}{2}T\,x_3 - T\,x_4 + x_{10}$ of Section 4.2; at $t = 1$ the bracket is the shared measurement and the rest reads $\frac{1}{2}\zeta_{\Pi_1 u} - \zeta_{\Pi_1\Pi_2 u}$ from group 2.

Step 4 (substitution). Variables receive schedules $a_v \in [2]^2$ in a balanced round robin; clauses receive $d_e \in [2]^2$ by a greedy choice that minimizes the maximum label multiplicity h; occurrence (e,v) gets label $u(e,v) = ((a_v(1), d_e(1)), (a_v(2), d_e(2)))$ and a copy $j(e,v) \in [h]$; copy j holds X_v at coordinate $u(e,v)$ and zero elsewhere; and $\vec{P}_{e,v}(T)$ is $\vec{P}_{u(e,v)}(T)$ applied to copy $j(e,v)$. The measurements are C applied to each copy, split by group into the rows of $C_{t,g}$ shown in Section 4.2. Every property is checked numerically as the objects are built, and condition (2.5) is verified at every point by solving for $\vec{\lambda}_{e,v,h}$.

B The exponent under the paper's parameters

With $\lambda = \lceil \log_2 n \rceil$ and $\nu = \lceil \log_2 \lambda \rceil$, the choices (2.19) give $R/Q \approx 4\sqrt{\lambda} \cdot 128\nu/\lambda = 512\nu/\sqrt{\lambda}$ and $\log_2(2Q/R) \approx \log_2\big(\sqrt{\lambda}/(256\nu)\big) = \frac{1}{2}\log_2\lambda - \log_2\nu - 8$, whose leading term is $\nu/2$; hence $T_1 = (R/Q)\log_2(2Q/R) \approx 256\,\nu^2/\sqrt{\lambda}$. Next $T_3 = 1/r \approx \nu^2/\sqrt{\lambda}$. Finally $T_2 = \tau \log_2(2D+1) \approx (1/(r\lambda)) \cdot 2Q \log_2 r \approx (\nu^2/\lambda^{3/2}) \cdot (\lambda/(64\nu)) \cdot \log_2 r$ with $\log_2 r \approx \frac{1}{2}\log_2\lambda - 2\log_2\nu$, leading term $\nu/2$, so $T_2 \approx \nu^2/(128\sqrt{\lambda})$. Thus $T_1 : T_3 : T_2 \approx 256 : 1 : 1/128$ to leading order, and deleting T_3 changes $T_1 + T_2 + T_3$ by a factor $1 - 1/257 + O(1/\nu)$. The lower-order terms we dropped ($-\log_2\nu - 8$ against $\nu/2$) are not small at any n one would write down, which is one more way of saying that the regime is asymptotic. With the integer choices of (2.19) the parameters are not even all defined for small λ: $r = \lfloor \sqrt{\lambda}/\nu^2 \rfloor$ is 0 at $n = 10^{100}$ and at $n = 10^{1000}$, and T_1 first becomes positive at $\lambda = 44{,}318{,}976 \approx 2^{25.4}$, so $\lambda = 2^{26}$ (that is, $n = 2^{2^{26}}$) is the smallest power of two with $T_1 > 0$. Evaluating the three terms exactly at $\lambda = 2^k$: $T_1/T_3 = 6, 48, 144, 196, 239$ and the share $T_3/(T_1+T_2+T_3)$ is $14.6\%, 2.0\%, 0.69\%, 0.51\%, 0.42\%$ for $k = 26, 32, 64, 128, 512$, against the limit 0.39%. The conclusion that T_1 dominates survives at every λ for which the terms are defined and positive; the figure "one part in 256" is the limit.